

Tipo di Dato Opaco

Puntatore a struct opaca e
modulo, signature, accoppiamento, coesione

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio

Digressione: Numeri Complessi

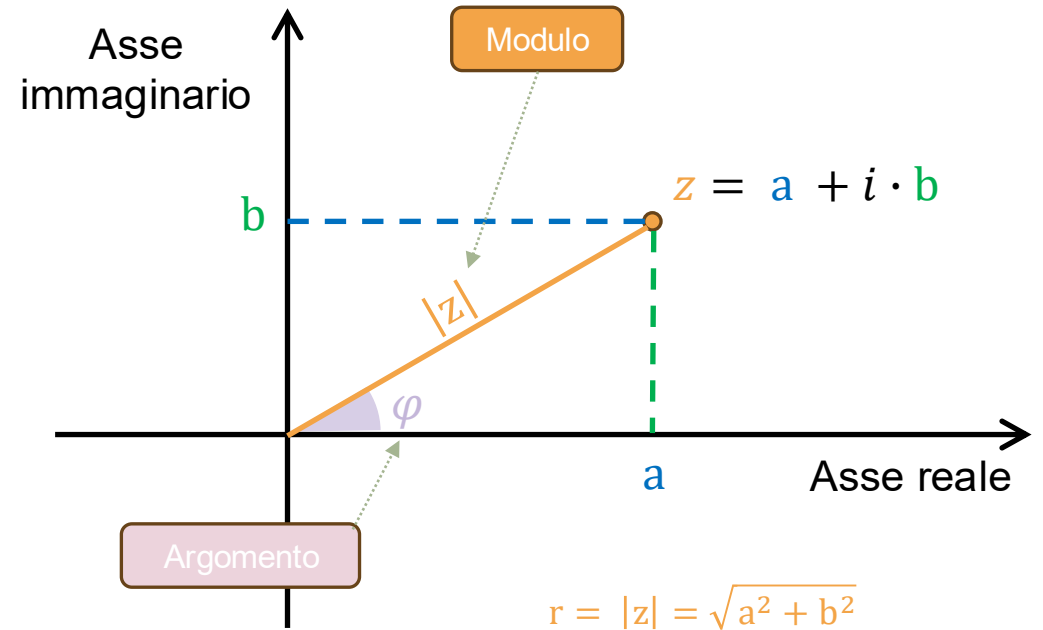
- Numeri complessi:

- Estensione dei numeri reali

$$z = a + i \cdot b$$

- Composti da due parti:

- Reale a e immaginaria b
- i è definita l'unità immaginaria
 - gode della proprietà: $i^2 = -1$



- Operazioni eseguite nel modo usuale ma sapendo che $i^2 = -1$:

- Somma:

- $z_1 + z_2 = (a_1 + i \cdot b_1) + (a_2 + i \cdot b_2) = (a_1 + a_2) + (b_1 + b_2) \cdot i$

- Prodotto:

- $z_1 z_2 = (a_1 + i \cdot b_1)(a_2 + i \cdot b_2) = (a_1 a_2 - b_1 b_2) + (b_1 a_2 + a_1 b_2) \cdot i$

$$\varphi = \begin{cases} \frac{\pi}{2} & \text{se } a = 0, b > 0 \\ -\frac{\pi}{2} & \text{se } a = 0, b < 0 \\ \arctan\left(\frac{b}{a}\right) & \text{se } a > 0 \\ \arctan\left(\frac{b}{a}\right) + \pi & \text{se } a < 0, b \geq 0 \\ \arctan\left(\frac{b}{a}\right) - \pi & \text{se } a < 0, b < 0 \\ \text{non definito} & \text{se } a = 0, b = 0 \end{cases}$$

Fonte e maggiori dettagli [qui](#).

Problema



- Supponiamo di dover realizzare un programma che tratta i numeri complessi implementando le seguenti operazioni:
 - somma (**sum**): dati due numeri complessi ne restituisce la somma
 - prodotto (**product**): dati due numeri complessi ne restituisce il prodotto
 - modulo (**modulus**): dato un numero complesso ne restituisce il modulo
 - argomento (**argument**): dato un numero complesso ne restituisce l'argomento, i.e., φ
 - uguali (**equal**): dati due numeri complessi restituisce un `_Bool` che dice se le rispettive parti reale e immaginarie sono uguali a meno di un $\epsilon \geq 0$
- Che tipo di dato definireste per rappresentare i numeri complessi?

Prima Soluzione I

- Numero complessi formati da due numeri reali
 - Idea: usiamo una struct con due double
- ... e definiamo i prototipi delle funzioni:

```
/** Tipo di dato per numeri complessi
 */
typedef struct {
    double re;
    double im;
} Dcomplex;
```

```
/** @brief Restituisce la somma di due complessi
 */
Dcomplex sum(Dcomplex c1, Dcomplex c2);

/** @brief Restituisce il prodotto di due complessi
 */
Dcomplex product(Dcomplex c1, Dcomplex c2);

/** @brief Restituisce il modulo di un complesso
 */
double modulus(Dcomplex c);

/** @brief Restituisce l'argomento nell'intervallo [-pi,+pi] radianti sempre definito
 */
double argument(Dcomplex c);

/** @brief Controlla se le componenti di due complessi sono uguali a meno di un epsilon
 */
_Bool equal(Dcomplex c1, Dcomplex c2, double epsilon);
```

Prima Soluzione II



- Applicando la metodologia TDD
 - Sviluppiamo un'implementazione `dcomplex.c` per le specifiche in `dcomplex.h` in modo iterativo e incrementale
 1. Definendo prima dei test in una test suite `test_dcomplex.c`
 2. Aggiornando il codice in `dcomplex.c`
 - ... finché tutte le specifiche sono testate e funzionanti

`test_dcomplex.c`

```
/** Una test suite per dcomplex.h
 */
#include "dcomplex.h"

int main(void) {
    /* Elenco di UNIT test per tutte le specifiche */
    // Omessi per brevità
}
```

`app.c`

```
/** Applicativo numeri complessi
 */
#include "dcomplex.h"

// Codice del app
```

Prima Soluzione: Implementazione



dcomplex.c

```
#include <math.h>
#include "dcomplex.h"
#define ABS(x) ((x) > 0) ? (x) : -(x)

Dcomplex sum(Dcomplex c1, Dcomplex c2)
{
    Dcomplex c = {};
    c.re = c1.re + c2.re;
    c.im = c1.im + c2.im;
    return c;
}

Dcomplex product(Dcomplex c1, Dcomplex c2)
{
    Dcomplex c = {};
    c.re = (c1.re * c2.re) - (c1.im * c2.im);
    c.im = (c1.re * c2.im) + (c1.im * c2.re);
    return c;
}
```

```
double modulus(Dcomplex c)
{
    return sqrt((c.re * c.re) + (c.im * c.im));
}

double argument(Dcomplex c)
{
    return atan2(c.im, c.re);
}

_Bool equal(Dcomplex c1, Dcomplex c2, double epsilon)
{
    return (ABS(c1.re - c2.re) <= epsilon) && (ABS(c1.im - c2.im) <= epsilon);
}
```

Prima Soluzione: Criticità?

- Quale criticità possiamo individuare nella specifica `dcomplex.h`?
 - Dcomplex è definito come tipo di dato trasparente
 - Espone la rappresentazione interna (i campi `re` e `im`)
 - Quindi, un qualunque codice che includa `dcomplex.h`, e.g., `app.c`:
 - Può contenere espressioni della forma `c.re` e `c.im`
 - Questo porta alla seguente criticità:
 - Cambiare `dcomplex.h` e `dcomplex.c`
 - Ad esempio per usare la rappresentazione polare, i.e., (r, φ) , al posto di quella cartesiana, i.e., (a, b)
 - comporta modificare **anche** `app.c` (e **tutti i file** che usano `dcomplex.h`)
 - Cioè:
 - La **signatura** `dcomplex.h` introduce un **accoppiamento** forte:
 - tra il **modulo** `app.c` e il **modulo** `dcomplex.c`

Qualche Definizione



- In ingegneria del software:
 - Un modulo (**module**) è un'unità di codice che può essere compilata separatamente
 - In C, è realizzabile con un file .c
 - La signature (**signature**) di un modulo dichiara la parte visibile di un modulo
 - In C, è realizzabile con un file .h
 - L'accoppiamento (**coupling**) esprime una misura dell'interdipendenza tra moduli
 - Misura quanto siano strettamente connessi due o più moduli; la forza delle relazioni tra i moduli.
 - La coesione (**cohesion**) esprime una misura della relazione tra le diverse funzioni presenti in un modulo e un qualche **concetto unificante** associato al modulo.
- Accoppiamento basso (**low coupling**) e coesione elevata (**high cohesion**) sono indici di un sistema informatico ben strutturato e di una buona progettazione
 - Supportano gli obiettivi generali di elevata leggibilità e manutenibilità del codice

Coesione di `dcomplex.h/c`



- Com'è la coesione di `dcomplex.h/c` ?
 - Implementa il tipo di dato "numero complesso"
 - È alta: "numero complesso" è il concetto unificante
- Se aggiungessimo a `dcomplex.h` una funzione con prototipo seguente?
 - `void printDcomplex(Dcomplex c)`
 - Coesione peggiora: mescoliamo i concetti di "numero complesso" e "stampa a video"
- E se invece aggiungessimo a `dcomplex.h` una funzione con prototipo?
 - `char *dctoa(Dcomplex c)` (restituisce una rappresentazione testuale di `c`)
 - Meglio, ma comunque meno coeso

Accoppiamento tra `dcomplex.h/c` e `app.c`

- Com'è l'accoppiamento tra `dcomplex.h/c` e `app.c`?
 - È alto dovuto all'uso di un tipo di dato trasparente
- È possibile abbassare l'accoppiamento?
 - Sì, usando un tipo di dato **opaco**



Tipo di Dato Opaco

Soluzione con Tipo Dato Opaco I

- Al fine di ridurre l'accoppiamento con i moduli client
 - Definiamo i numeri complessi come un tipo di dato **opaco** in `ADTcomplexOps.h`
 - Implementa un **Abstract Data Type (ADT)**

- Come?

- Sfruttando:
 - La memoria dinamica
 - Il fatto che in C si può usare il tipo

`struct label *`

in un file in cui la dichiarazione di `struct label` non è visibile.

`ADTcomplexOps.h`

```
/** Un tipo di dato opaco per i numeri complessi
 */

typedef struct dcomplex *ADTcomplex; /* puntatore a
struttura opaca */
```

Soluzione con Tipo Dato Opaco II

- E consideriamo in aggiunta* anche le operazioni:
 - **mkADTcomplex**: date parte reale e immaginaria alloca un numero complesso sullo heap
 - **re**: dato un numero complesso ne restituisce la parte reale
 - **im**: dato un numero complesso ne restituisce la parte immaginaria
 - **dsADTcomplex**: dato un numero complesso lo distrugge (deallocando la memoria)

*Operazioni definite in precedenza: **sum**, **product**, **modulus**, **argument**, **equal**

ADTcomplexOps.h



```
/** Un tipo di dato opaco per i numeri complessi
 */
typedef struct dcomplex *ADTcomplex; /* puntatore a struttura opaca */

/** @brief alloca un numero complesso a partire da parte
 * reale e immaginaria
 */
ADTcomplex mkADTcomplex(double re, double im);

/** @brief libera la memoria puntata da *`pc' e setta *`pc' a NULL
 */
void dsADTcomplex(ADTcomplex *pc);

/** @brief Restituisce la somma di due complessi
 */
ADTcomplex sum(ADTcomplex c1, ADTcomplex c2);

/** @brief Restituisce la prodotto di due complessi
 */
ADTcomplex product(ADTcomplex c1, ADTcomplex c2);

/** @brief Restituisce il modulo di un complesso
 */
double modulus(ADTcomplex c);
```

```
/** @brief Restituisce l'argomento nell'intervallo [-pi,+pi] radianti
 * N.B.: sempre definito, può essere -pi (dove pi non è il valore esatto di "pi greco")
 */
double agument(ADTcomplex c);

/** @brief Restituisce la parte reale
 */
double re(ADTcomplex c);

/** @brief Restituisce la parte immaginaria
 */
double im(ADTcomplex c);

/** @brief Controlla se le componenti di due complessi sono uguali a meno di un
 epsilon reale
 */
_Bool equal(ADTcomplex c1, ADTcomplex c2, double epsilon);
```

Soluzione con Tipo Dato Opaco III



- Applicando la metodologia TDD
 - Sviluppiamo un'implementazione `ADTcomplexCartesian.c` per le specifiche in `ADTcomplexOps.h`
 - Partendo da una test suite `test_ADTcomplexOps.c`
 - Test tralasciati sulle slide per brevità

`test_ADTcomplexOps.c`

```
/** Una test suite per ADTcomplexOps.h
 */
#include "ADTcomplexOps.h"

int main(void) {
    /* Elenco di UNIT test per tutte le specifiche */
    // Omessi per brevità
}
```

`app_v2.c`

```
/** Applicativo numeri complessi
 */
#include "ADTcomplexOps.h"

// Codice del app
```

ADTcomplexCartesian.c



```
#include <stdlib.h>
#include <math.h>
#include "ADTcomplexOps.h"

#define ABS(x) ((x) > 0) ? (x) : -(x)

struct dcomplex
{
    double re;
    double im;
};

ADTcomplex mkADTcomplex(double re, double im) {
    ADTcomplex c;
    c = malloc(sizeof(struct dcomplex));
    c->re = re; c->im = im;
    return c;
}

void dsADTcomplex(ADTcomplex *pc) {
    free(*pc);
    *pc = NULL;
}

ADTcomplex sum(ADTcomplex c1, ADTcomplex c2) {
    return mkADTcomplex(c1->re + c2->re,
                        c1->im + c2->im);
}
```

```
ADTcomplex product(ADTcomplex c1, ADTcomplex c2) {
    return mkADTcomplex((c1->re * c2->re) - (c1->im * c2->im),
                        (c1->re * c2->im) + (c1->im * c2->re));
}

double modulus(ADTcomplex c) {
    return sqrt((c->re * c->re) + (c->im * c->im));
}

double argument(ADTcomplex c) {
    return atan2(c->im, c->re);
}

double re(ADTcomplex c) {
    return c->re;
}

double im(ADTcomplex c) {
    return c->im;
}

_Bool equal(ADTcomplex c1, ADTcomplex c2, double epsilon) {
    return (ABS(c1->re - c2->re) <= epsilon)
        && (ABS(c1->im - c2->im) <= epsilon);
}
```


Confronto I



- Cosa possiamo fare con `ADTcomplexOps.h` (che non potevamo fare con `dcomplex.h`)?
- Queste cose potevamo farle anche con `dcomplex.h`:
 - Compilare separatamente il file `app_v2.c` (generando un file `app_v2.o`)

```
gcc -c app_v2.c
```
 - Linkare (collegare) i file `app.o` e `ADTcomplexCartesian.o` (che qualcuno avrà generato compilando il sorgente del lucido precedente) generando il file eseguibile `app`:

```
gcc -o app app_v2.o ADTcomplexCartesian.o
```

Confronto II



- Cosa possiamo fare con `ADTcomplexOps.h` (che non potevamo fare con `dcomplex.h`)?
- Questa cosa invece non potevamo farla prima
 - Cambiare l'implementazione `ADTcomplexCartesian.o` con `ADTcomplexPolar.o` (che qualcuno avrà generato compilando il sorgente del lucido successivo)
 - Senza modificare `ADTcomplexOps.h`
 - E senza ricompilare file `app_v2.c`
 - Limitandoci a lanciare il comando:

```
gcc -o app app_v2.o ADTcomplexPolar.o
```

ADTcomplexPolar.c



```
#include <stdlib.h>
#include <math.h>
#include "ADTcomplexOps.h"
#define ABS(x) ((x) > 0) ? (x) : -(x)

struct dcomplex
{
    double modulus;
    double argument;
};

ADTcomplex mkADTcomplex(double re, double im) {
    ADTcomplex c;
    c = malloc(sizeof(struct dcomplex));
    c->modulus = sqrt((re * re) + (im * im));
    c->argument = atan2(im, re);
    return c;
}

void dsADTcomplex(ADTcomplex *pc) {
    free(*pc);
    *pc = NULL;
}

ADTcomplex sum(ADTcomplex c1, ADTcomplex c2) {
    return mkADTcomplex(re(c1) + re(c2), im(c1) + im(c2));
}
```

```
ADTcomplex product(ADTcomplex c1, ADTcomplex c2) {
    ADTcomplex c = malloc(sizeof(struct dcomplex));
    c->modulus = c1->modulus * c2->modulus;
    c->argument = c1->argument + c2->argument;
    return c;
}

double modulus(ADTcomplex c) {
    return c->modulus;
}

double argument(ADTcomplex c) {
    return c->argument;
}

double re(ADTcomplex c) {
    return c->modulus * cos(c->argument);
}

double im(ADTcomplex c) {
    return c->modulus * sin(c->argument);
}

_Bool equal(ADTcomplex c1, ADTcomplex c2, double epsilon) {
    return (ABS(re(c1) - re(c2)) <= epsilon) && (ABS(im(c1) - im(c2)) <= epsilon);
}
```

Criticità di `ADTcomplexOpq.h`



- Quali criticità possiamo individuare nella specifica `ADTcomplexOps.h`?
 1. Il programmatore deve gestire esplicitamente allocazione (con `mkADTcomplex`) e deallocazione (con `dsADTcomplex`) della memoria:
 - Pericolo di **memory leak**
 - Se si dimentica di distruggere dei valori `ADTcomplex` che non servono più
 - Un memory leak (“perdita o fuoriuscita di memoria”) è un particolare tipo di consumo non voluto di memoria dovuto alla mancata deallocazione dalla stessa di variabili o dati non più utilizzati.
 - Pericolo di **risultati sbagliati e casuali**
 - Se uso due variabili `c1` e `c2` di tipo `ADTcomplex` che contengono lo stesso valore (ovvero puntano allo stesso blocco di memoria)
 - Invocando `dsADTcomplex(c1)`, `c1` è cambiato in `NULL`, ma non `c2`
 - Se continuo a usare `c2`, allora il programma esibirà degli errori strani
 2. Le chiamate a `malloc` e `free` sono operazioni lente

Compromesso tra
`dcomplex.h` e
`ADTcomplexOpq.h` ?

Compromesso tra
`dcomplex.h` e
`ADTcomplexOps.h`

Compromesso tra `dcomplex.h` e `ADTcomplexOpq.h`

- Per realizzare un tipo di dato che prende i pregi (ma non i difetti) di `dcomplex.h` e `ADTcomplexOps.h` spezziamo la sua signature in due file:
 - `dcomplexRep.h`
 - Dove il suffisso “Rep” sta per “Representation” (“rappresentazione”, in Italiano)
 - Questa è la parte della specifica che definisce la rappresentazione del dato
 - `dcomplexAgn.h`
 - Dove il suffisso “Agn” sta per “Agnostic” (“agnostico”, in Italiano)
 - Questa è la parte in cui, sebbene la rappresentazione del dato sia inclusa nella specifica (e quindi visibile al codice client), il codice client non ne fa uso e quindi è agnostico rispetto alla sua rappresentazione.

dComplexRep.h e dComplexAgn.h

dcomplexRep.h

```
/** Un (quasi) ADT per i numeri complessi rappresentati in coordinate
cartesiane
* -- NON ACCEDERE AI CAMPI DI QUESTA struct NEL CODICE CLIENT
*/
typedef struct {
    double re;
    double im;
} Dcomplex;
```

dcomplexAgn.h

```
/** Un tipo di dato per i numeri complessi
*/
#include "dcomplexRep.h"

Dcomplex mkDcomplexCar(double re, double im);

Dcomplex mkDcomplexPol(double modulus, double argument);

Dcomplex sum(Dcomplex c1, Dcomplex c2);

Dcomplex product(Dcomplex c1, Dcomplex c2);

double modulus(Dcomplex c);

double agument(Dcomplex c);

double re(Dcomplex c);

double im(Dcomplex c);

_Bool equal(Dcomplex c1, Dcomplex c2, double epsilon);
```



Soluzione Agnostica

- Applicando la metodologia TDD
 - Sviluppiamo un'implementazione `dComplexCarOpt.c` per le specifiche in `dComplexAgnOpt.h`
 - Partendo da una test suite `test_dComplexAgnOpt.c`
 - Test tralasciati sulle slide per brevità

`test_dComplexAgn.c`

```
/** Una test suite per dComplexAgn.h
 */
#include "dComplexAgn.h"

int main(void) {
    /* Elenco di UNIT test per tutte le specifiche */
    // Omessi per brevità
}
```

`app_v3.c`

```
/** Applicativo numeri complessi
 */
#include "dcomplexAgn.h"
// Codice del app
```

⚠️ Includo la parte agnostica

dComplexCar.c

Perché abbiamo
usato `static` ?

```
#include <math.h>
#include "dcomplexAgn.h"

static double dabs(double x) { return (x > 0) ? x : -x; }

Dcomplex mkDcomplexCar(double re, double im) { Dcomplex c; c.re = re; c.im = im; return c; }

Dcomplex mkDcomplexPol(double modulus, double argument) { return mkDcomplexCar(modulus * cos(argument), modulus * sin(argument)); }

Dcomplex sum(Dcomplex c1, Dcomplex c2) { return mkDcomplexCar(c1.re + c2.re, c1.im + c2.im); }

Dcomplex product(Dcomplex c1, Dcomplex c2) { return mkDcomplexCar((c1.re * c2.im) - (c1.im * c2.re), (c1.re * c2.im) + (c1.im * c2.re)); }

double modulus(Dcomplex c) { return sqrt((c.re * c.re) + (c.im * c.im)); }

double agument(Dcomplex c) { return atan2(c.im, c.re); }

double re(Dcomplex c) { return c.re; }

double im(Dcomplex c) { return c.im; }

_Bool equal(Dcomplex c1, Dcomplex c2, double epsilon) { return (dabs(c1.re - c2.re) <= epsilon) && (dabs(c1.im - c2.im) <= epsilon); }
```



Soluzione con `dComplexAgn.h` I

- Possiamo usare `dcomplexAgn.h` nello stesso modo di `dcomplex.h`
 1. Compiliamo separatamente il file `app_v3.c` (generando `app_v3.o`)

```
gcc -c app_v3.c
```
 2. Linkiamo (colleghiamo) i file `app_v3.o` e `dComplexCar.o` (che qualcuno avrà generato compilando il sorgente del lucido precedente):

```
gcc -o app app_v3.o dComplexCar.o (generando l'eseguibile app)
```

Soluzione con dComplexAgn.h II

- Se poi vogliamo cambiare implementazione usando la rappresentazione polare:

1. Dobbiamo modificare la definizione della struct nel file dComplexRep.h
2. Ricompilare app_v3.c e linkare app_v3.o con dComplexPol.o (che qualcuno avrà generato compilando il sorgente dComplexPol.c):

```
gcc -c app_v3.c
gcc -o app app_v3.o dComplexPol.o
```

POLAR permette di scegliere la rappresentazione da usare

dComplexRep.h

```
/** Un (quasi) ADT per i numeri complessi rappresentati in
coordinate cartesiane o polari
* -- NON ACCEDERE AI CAMPI DI QUESTA struct NEL CODICE
CLIENT
*/
#define POLAR

typedef struct {
#ifdef POLAR
    double modulus;
    double argument;
#else
    double re;
    double im;
#endif
} Dcomplex;
```

Esercizio

- Sviluppate il codice di `dComplexPol.c`

`dComplexPol.c`

```
#include <math.h>
#include "dcomplexAgn.h"

static void dabs(double x) { return (x > 0) ? x : -x; }

/* Qui ometto il mirabile codice che dovrete scrivere a voi.
*/
```



Criticità Tutte Risolte?

- Usando `dComplex.h/c`
 - `app.c` può accedere direttamente ai membri della `struct`, inoltre, il calcolo del valore assoluto è implementato come una macro.
- Invece, usando `dComplexAgn.h`, `dComplexCar.c`/`dComplexPol.c`
 - `app_v3.c` deve usare delle funzioni per accedere ai membri della `struct`, inoltre il calcolo del valore assoluto è implementato come una funzione
- Con `dComplexAgn.h` si scrive codice meno efficiente?
 - No, i moderni compilatori ottimizzano facendo l'inlining* delle funzioni piccole

*L'inlining è un'ottimizzazione che sostituisce un sito di chiamata di funzione con il corpo della funzione chiamata. L'inlining è simile all'espansione macro, ma si verifica durante la compilazione, senza modificare il codice sorgente (il testo), mentre l'espansione macro si verifica prima della compilazione.

Criticità Tutte Risolte?

- Quando si modifica l'implementazione (e.g., da cartesiana a polare):
 - Si deve modificare anche (la configurazione di) `dcomplexRep.h`
 - Quindi il codice client (che include `dcomplexAgn.h` che a sua volta include `dcomplexRep.h`) deve essere ricompilato
- **Invece usando `ADTcomplexOps.h` e `ADTcomplexPolar.c`**
 - Non sarebbe necessario modificare `ADTcomplexOps.h`
 - Quindi si potrebbe fare a meno di ricompilare il codice cliente
- Abbiamo aumentato l'accoppiamento?
 - Sì, ma a livello di build dell'applicazione, non a livello di codice sorgente
 - Il codice cliente non richiede modifiche

Codice Efficiente

struct Passati per Valore



- In `dComplexAgn.h` gli argomenti delle funzioni `sum` e `product` sono passati per valore
 - Passandoli per riferimento si risparmierebbero spazio e tempo
 - In questo caso circa il 50%, ma se la struct fosse più grande il costo del passaggio per valore potrebbe diventare un problema
- Possiamo scrivere codice più efficiente?
 - Sì, passando i paramtri per riferimento
 - Usiamo `const` per i parametri che non devono esser modificati

dComplexAgnOpt.h

Argomenti struct
passati per riferimento

dComplexAgnOpt.h

```
/** Un (quasi) ADT per i numeri complessi
 * -- NON ACCEDERE ALLA RAPPRESENTAZIONE DEL TIPO Dcomplex NEL CODICE CLIENT
 */
#include "dComplexRep.h"

Dcomplex mkDcomplexCar(double re, double im);

Dcomplex mkDcomplexPol(double modulus, double argument);

Dcomplex sum(const Dcomplex *const pc1, const Dcomplex *const pc2);

Dcomplex product(const Dcomplex *const pc1, const Dcomplex *const pc2);

double argument(const Dcomplex *const pc);

double re(const Dcomplex *const pc);

double im(const Dcomplex *const pc);

_Bool equal(const Dcomplex *const pc1, const Dcomplex *const pc2, double epsilon);
```



Soluzione Agnostica Ottimizzata



- Applicando la metodologia TDD
 - Sviluppiamo un'implementazione `dComplexCarOpt.c` per le specifiche in `dComplexAgnOpt.h`
 - Partendo da una test suite `test_dComplexAgnOpt.c`
 - Test tralasciati sulle slide per brevità

`test_dComplexAgnOpt.c`

```
/** Una test suite per dComplexAgnOpt.h
 */
#include "dComplexAgnOpt.h"

int main(void) {
    /* Elenco di UNIT test per tutte le specifiche */
    // Omessi per brevità
}
```

`app_v4.c`

```
/** Applicativo numeri complessi
 */
#include "dcomplexAgnOpt.h"
// Codice del app
```

dComplexCarOpt.c

```
#include <math.h>
#include "dComplexAgnOpt.h"

static double dabs(double x) { return (x > 0) ? x : -x; }

Dcomplex mkDcomplexCar(double re, double im) { Dcomplex c; c.re = re; c.im = im; return c; }

Dcomplex mkDcomplexPol(double modulus, double argument) { return mkDcomplexCar(modulus * cos(argument), modulus * sin(argument)); }

Dcomplex sum(const Dcomplex *const pc1, const Dcomplex *const pc2) { return mkDcomplexCar(pc1->re + pc2->re, pc1->im + pc2->im); }

Dcomplex product(const Dcomplex *const pc1, const Dcomplex *const pc2) {
    return mkDcomplexCar((pc1->re * pc2->im) - (pc1->im * pc2->re), (pc1->re * pc2->re) + (pc1->im * pc2->im));
}

double modulus(const Dcomplex *const pc) { return sqrt(((pc->re) * (pc->re)) + ((pc->im) * (pc->im))); }

double argument(const Dcomplex *const pc) { return atan2(pc->im, pc->re); }

double re(const Dcomplex *const pc) { return pc->re; }

double im(const Dcomplex *const pc) { return pc->im; }

_Bool equal(const Dcomplex *const pc1, const Dcomplex *const pc2, double epsilon) {
    return (dabs(pc1->re - pc2->re) <= epsilon) && (dabs(pc1->im - pc2->im) <= epsilon);
}
```



Esercizio

- Sviluppate il codice di `dComplexPolOpt.c`

`dComplexPolOpt.c`

```
#include <math.h>
#include "dcomplexAgnOpt.h"

static void dabs(double x) { return (x > 0) ? x : -x; }

/* Qui ometto il mirabile codice che dovrete scrivere a voi.
*/
```



Possiamo Migliorare Ulteriormente?

struct Ritornati per Valore

- In `dcomplexAgnOpt.h` i risultati delle funzioni `mkDcomplexCar`, `mkDcomplexPol`, `sum` e `product` sono struct
 - Se nella maggior parte dei casi, questi risultati sono assegnati a variabili nel codice dell'applicativo possiamo migliorare l'efficienza passando tali variabili per riferimento e modificandole all'interno della funzione
 - In questo caso circa il 50% di risparmio in tempo e spazio, ma se la struct fosse più grande il risparmio potrebbe essere maggiore
- Riusciamo a migliorare ulteriormente l'efficienza del codice?

dComplexAgnOpt2.h

- La specifica dComplexAgnOpt2.h è ottenuta dalla specifica dComplexAgnOpt.h aggiungendo 4 prototipi

```
#include "dComplexAgnOpt.h"

void initDcomplexCar(Dcomplex *pc, double re, double im);

void initDcomplexPol(Dcomplex *pc, double modulus, double argument);

void sum3(Dcomplex *pc, const Dcomplex *const pc1, const Dcomplex *const pc2);

void product3(Dcomplex *pc, const Dcomplex *const pc1, const Dcomplex *const pc2);
```

Modifichiamo direttamente
il valore in memoria

Soluzione Agnostica Ottimizzata V2



- Applicando la metodologia TDD
 - Sviluppiamo un'implementazione `dComplexCarOpt2.c` per le specifiche in `dComplexAgnOpt2.h`
 - Partendo da una test suite `test_dComplexAgnOpt2.c`
 - Test tralasciati sulle slide per brevità

`test_dComplexAgnOpt2.c`

```
/** Una test suite per dComplexAgnOpt2.h
 */
#include "dComplexAgnOpt2.h"

int main(void) {
    /* Elenco di UNIT test per tutte le specifiche */
    // Omessi per brevità
}
```

`app_v5.c`

```
/** Applicativo numeri complessi
 */
#include "dcomplexAgnOpt2.h"
// Codice del app
```


dComplexAgnOpt2.c



- Riportiamo solo le 4 funzioni nuove di dComplexAgnOpt2.c

```
void initDcomplexCar(Dcomplex *pc, double re, double im) { pc->re = re; pc->im = im; }

void initDcomplexPol(Dcomplex *pc, double modulus, double argument) { pc->re = modulus*cos(argument); pc->im = modulus*sin(argument); }

void sum3(Dcomplex *pc, const Dcomplex *const pc1, const Dcomplex *const pc2) {
    pc->re = pc1->re + pc2->re;
    pc->im = pc1->im + pc2->im;
}

void product3(Dcomplex *pc, const Dcomplex *const pc1, const Dcomplex *const pc2) {
    pc->re = (pc1->re * pc2->im) - (pc1->im * pc2->re);
    pc->im = (pc1->re * pc2->re) + (pc1->im * pc2->im);
}
```

Esercizio

- Sviluppate il codice di `dComplexPolOpt2.c`

`dComplexPolOpt2.c`

```
#include <math.h>
#include "dcomplexAgnOpt2.h"

static void dabs(double x) { return (x > 0) ? x : -x; }

/* Qui ometto il mirabile codice che dovrete scrivere a voi.
*/
```



Soluzione Agnostica Ottimizzata V2: Commenti



- Perché in `dcomplexAgnOpt2.h` non abbiamo tolto le 4 funzioni `mkDcomplexCar`, `mkDcomplexPol`, `sum` e `product`?
 - Perché, in alcuni casi, sono comode da usare:
 - Ad esempio per scrivere espressioni in cui i valori intermedi non devono essere salvati in nessuna variabile:
 - `sum(mkDcomplexCar(4,7), mkDcomplexCar(9,3))`
 - `product(x, sum(y,z))`
- Però attenzione all'efficienza!

Commenti Finali



- Progettate in termini di **moduli** e **signature** (dei moduli): sono concetti generali
 - In C si incarnano nei files .c e .h, rispettivamente
 - In altri linguaggi si incarnano in altri modi.
- Progettate moduli e signature cercando di ottenere:
 - **Accoppiamento basso** e **coesione alta**
 - Puntatori a strutture opache supportano l'accoppiamento basso
 - Ma richiedono un prezzo da pagare e **non sempre è necessario/opportuno pagarlo**
- Codice efficiente:
 - Talune ottimizzazioni sono fatte dai moderni compilatori (dovete sapere quali, per non investirci inutilmente tempo), talaltre le devono fare i programmatori.